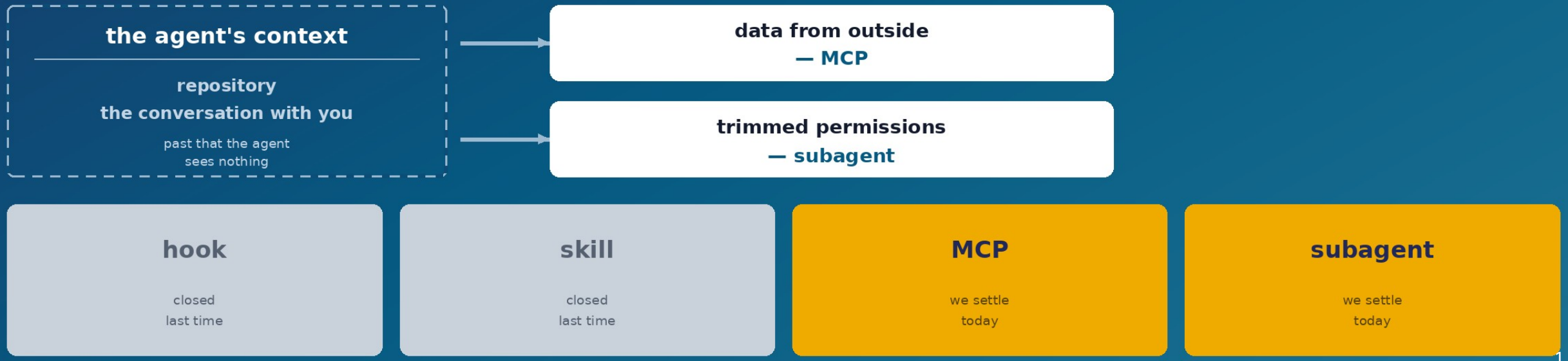
Access to the outside and trimmed permissions

> Two weeks ago the agent knew everything about this project. On Monday it answered confidently and wide of the mark.

"The agent did not get dumber — so what stopped fitting?"



Where we left off

RUNG	PATH OR ARTIFACT	STATUS
Hook	`.claude/settings.json` + `.claude/hooks/selftest-branch-guard.sh`	has a commit
Skill	`.claude/skills/deploy/SKILL.md`	only a plan, no commit
CLAUDE.md	was 24 lines / 4 sections → now 40 lines / 5 sections	has a commit
DECISIONS.md	the repository's documentation layer	no change

"Four things, and all four of them inside the repository. As long as the work was here, the agent held the project whole."

Two rows closed — both about what is inside

RUNG	PREVENT / SETTLE	WHAT IT DOES	WHAT IT DOES NOT DO / THE LIMIT
Hook	Settle — the rule has already been broken	Rejects one pattern of writing into `main` out of six ways of committing	On the other five it is silent; a person with a terminal it does not see at all
Skill	Settle — on a signal that the instruction file is overloaded	Loads a file on an exact match of the trigger description	Without an exact trigger it will not fire at all — the file's contents are not in the context until it fires
MCP			
Subagent			

Hooks and skills work inside the repository. Both of today's rows are about its edge.

Two misses, opposite causes

WHEN AND WHAT	WHY IT CAME OUT THAT WAY
Monday. It answered by the repository — two of the client's requests were not in the answer. A missed deadline.	They are not inside at all. Let the context be twice the size — they still will not appear in it.
Wednesday, a working product. It read everything where the rule might be checked, assembled a correct list — and then started re-asking what had been explained at the start of the session. Half an hour.	Everything needed fit inside — and crowded out the very thing the session was opened for.

"The work has run past the agent's context — which do you set up first: bringing inside what is not inside, or carrying outside what does not fit inside?"

Two sections — two answers to one question

SECTION	FIRST	SECOND	THIRD
Bring inside what is not inside — MCP, access to the outside	The agent regularly needs data that is not in the repository. We settle it — and check that the agent really does go through the server.	Connected servers take up context before the work begins. We settle it.	—
Carry outside what does not fit inside — subagent, trimmed permissions	The reading needed for one edit crowds the edit itself out of the context. We settle it.	The task arrived as a single sentence, and the deadline on it is two days off. We settle it.	There are several workers, and they have one working folder. We settle it.

"One and the same limit. Two moves running toward each other."

1

"But I did write that into the issue"

The task lives in the tracker and changes without us. What the agent has in the chat is a snapshot taken by hand this morning.

MCP

Subagent

Files inside, files outside — and the issue tracker?

WHAT THE AGENT ALREADY USES

The repository's files — the agent reads them directly, this is the material of previous classes.

Files handed over explicitly: text pasted into the chat, a document attached.

Both sources are files. Read it and get to work.

AND THE LIST OF ISSUES IN THE TRACKER?

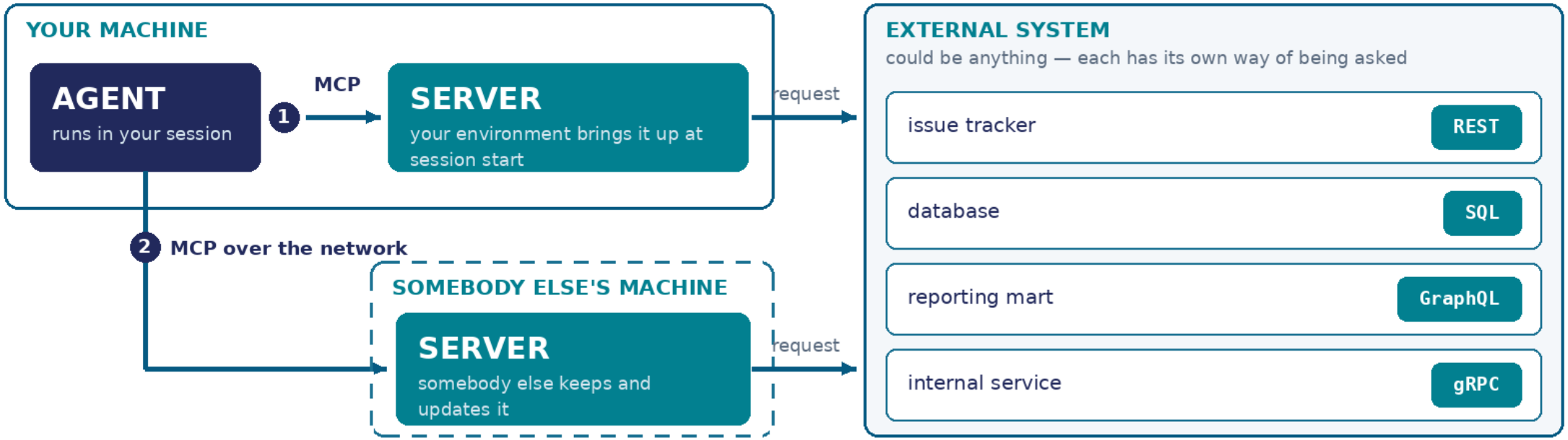
The client keeps it apart from the project: there is no such file in the repository's tree.

Handing it over once and forgetting about it will not work — the list changes without us every day.

A snapshot as of this morning will be out of date by lunchtime. What is needed is a way to ask the system at the moment of the work.

Reaching a system like that is called an API call. The usual guess at this point: you will have to pay for it with an integration you write. We will check that against five ways.

The server — at your place or outside, the conversation is the same



WHO TALKS TO WHOM

The agent talks only to the server, the same way in both variants. It is the server that talks to the external system — it holds the key.

WHAT COMES BACK

The agent gets back the system's answer as text — exactly what the server handed over, never touching the system.

One server can reach several systems — each its own way: REST, gRPC, GraphQL, SQL.
The agent does not know those ways: to any server it has one conversation — MCP.

MCP and the server, in one screen

1 WHAT A SERVER IS

A separate program outside Claude Code. It answers requests by the protocol.

stdio — local Claude Code starts the process on your machine

http — remote a service that somebody else hosts and updates

The command in the config runs literally at session start: whatever is written is what starts.

2 WHERE DECLARED

```
{ "mcpServers": { "github": {
      "command": "npx", … } } }
```

`.mcp.json` — in the repository — the project scope

`~/.claude.json` — local (default) and user scopes

which scope and why — broken down later in the case

3 WHAT HAPPENS ON CONNECTION

The transport comes up — Claude Code ITSELF, with no model, asks the server for its list:

`tools/list` `prompts/list` `resources/list`

By default, only the tool names and server instructions land in the context from this answer; full schemas come later, on demand. The deferral is cancelled by a flag on an individual server and by an environment variable — then all schemas load at start.

4 CALL: AGENT ASSEMBLES THE PARAMETERS

```
mcp__github__create_issue
{ title, body, … }
```

The agent itself brings up the server and calls it — like calling a function: the full name with the server's prefix, and arguments the model assembled.

5 CONTEXT & COST — THREE LINE ITEMS

session start tool names and the server's instructions — always in the context

by default the model pulls in the tool's full schema on demand

reply arrived the call's result — a separate budget: a warning at 10,000 tokens, truncated at 25,000

The standing cost is paid for a connected server. The cost of a call is separate, per answer.

6 WHAT A SERVER ANSWERS WITH

The result arrives over a separate channel from capability discovery — not the one used on connection.

```
tool call
mcp__github__create_issue
```

The call is signed with the server's name — visibly from there, not the model's general knowledge.

The Tuesday the transfer by hand failed

The site is in its third month. For a month now the client has been keeping his tasks in a tracker — the developer suggested it himself, so that edits would stop getting lost in a messenger. The transfer between the tracker and the agent's chat the developer does by hand, a few times a week.

1. Tuesday morning: opened the issue "the form does not submit from a phone", copied the text into the agent's chat, went off to make the edit.
2. In the afternoon the client added a line to that same issue: "and from a tablet too".
3. The developer did not see it — what he had in the chat was the morning's snapshot. In the evening the edit went out to the client.
4. In the morning, in the issue: "what about the tablet?". An hour on a second edit and an email explaining why a requirement from the issue did not make it through to the work.

The hour lost is the smaller part of the cost. The larger part is that the client now double-checks whether what he wrote reached the work. The tracker lives in a separate system and changes every day without the developer: a snapshot taken in the morning is already inaccurate by evening.

How do you give the agent access to the issue tracker?

Which of you has already had an agent go outside — into a tracker, into a database, into documentation? What did you give it access with back then?

Pick your own way out of the five — and say whether it stands up to regular access, or whether you would take the card next to it now.

- call the API directly
- connect somebody else's server — remotely
- connect somebody else's server — locally
- write your own server
- call the ready-made gh command every time

Not one of the five writes integration code

WAY	WHAT IT DOES	WHAT IT DOES NOT DO / THE LIMIT
call the API directly	the agent writes and sends the requests to the endpoints itself and parses the answer itself — no separate integration code needs to be written	it does not know the service's specification in advance: which points there are, what their parameters and limits are — without documentation or a Swagger description the agent will not guess them
connect somebody else's server — remotely	a service somebody hosts and updates themselves; no separate process runs at your place	you are not the one managing it: there is no seeing what the service does inside or when something in it changes; plus the ordinary security questions of a remote service — whom you trust the key to, who else has access
connect somebody else's server — locally	the package runs at your place, the configuration is a line in the repository, visible to the whole team in review; more control than with a remote one	it is still a live process on your machine and somebody else's code — with all of its dependencies and its right to update itself
write your own server	gives exactly the set of operations and permissions needed, with no outside code in the process	writing a minimal server is quick these days — there is a ready-made plugin for it; what comes expensive is maintenance: the protocol, outside users, the security of the transport — a separate project alongside the main one
call the `gh` command every time	GitHub's ready-made official CLI — it has already solved authorization and parsing the answer for you, no code needs writing	every call is a separate command with no shared list of capabilities: the agent does not see in advance what can be done at all, and the call it needs has to be known in advance

The turn. No integration gets written in any of the five ways: the agent sends the request itself and parses the answer itself. What separates the five is something else — whether the agent knows in advance what the system can be asked.

What that leaves. Your own server is excessive: an official one for the tracker already exists. `gh` and a direct API call cover a call that is known in advance; a growing list of operations they offload onto the developer's memory.

Two legitimate variants. Between "remotely" and "locally" there is no ready answer: the latest official example connects remotely, and the seminar's decision is a local process, because the configuration then becomes code in the repository, visible in review.

We settle it: which operations to allow, then which permissions

Move 1 — what specifically does the agent have to do?

- read open issues
- write a comment
- create a draft edit

Move 2 — what does a broad personal access token give by default, and which of it is needed?

DEFAULT PERMISSION	NEEDED FOR OUR LIST OF OPERATIONS?
all of the user's repositories, including the private ones	no — cross it out
read and write	in part — reading is needed, and writing (a comment, a draft) is needed
managing settings	no — cross it out
deletion	no — cross it out

What remains: one repository, reading issues, creating a draft edit.

We settle it: where to keep the key and which scope to put the file in

The key goes in an environment variable; the config file, in one of three scopes

Move 3 — where to keep the key? an environment variable, not a file value

Move 4 — where to put the file?

`local`

only on my own machine

an unchecked experiment

the connect command default

`project`

into the repository, for all

a checked server — the configuration becomes code, change shows in review

`user`

globally, across all my projects

personal tools, not specific to the project

The connect command default puts the server into local, not project. Not knowing that gets you "I connected it, it works" — "well, not for me", and both are right.

Why the boxes get clicked straight through — and what comes next

A key's permissions are asked about one single time — at the moment of issue: the service lists the actions and asks which of them you trust to this key.

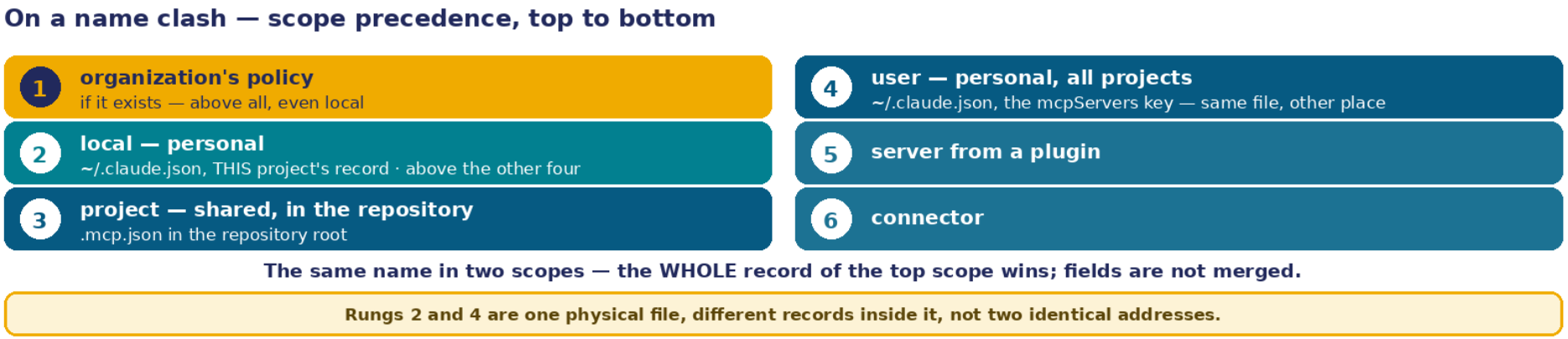
THE MOVE ALMOST EVERYONE MAKES	WHAT IN IT COMES EXPENSIVE
tick every box in a row: "what if I need it later, surely I am not going to reissue the key"	the agent's list of operations is three lines long, and the key was issued for all of them: what was crossed out in the breakdown came back in its entirety
put the key within reach — into a file next to the configuration	the file travels into the repository, into the commit history and into the review of edits. The key's home between runs is a secrets store: the name is visible, the value is not
count on nobody looking for your key	keys in public repositories do get searched for and do get found working; one of the model providers watches over its own and blocks an exposed one within seconds — by the account of students from a previous cohort
remember about permissions once the key has already surfaced somewhere	permissions are changed on the service's side, and after the fact they cancel nothing: whatever the key managed, it managed

The provider's watchdog covers one channel out of many — a private leak, a build log and a screenshot it does not see. A narrow key that has leaked is all that remains within your power after it is issued.

The assembled file — and the order on a clash of names

```
{
  "mcpServers": {
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": { "GITHUB_PERSONAL_ACCESS_TOKEN": "${GITHUB_PAT}" }
    }
  }
}
```

- `mcpServers` — the root: all the servers of this scope. `"github"` — the server's name: it is the name the agent calls the tools under, `mcp__github__…`.
- `"command"` and `"args"` — what starts it and what exactly gets started: the server's own package.
- `"env"` — the process's environment; `${GITHUB_PAT}` is a substitution, not the secret itself.



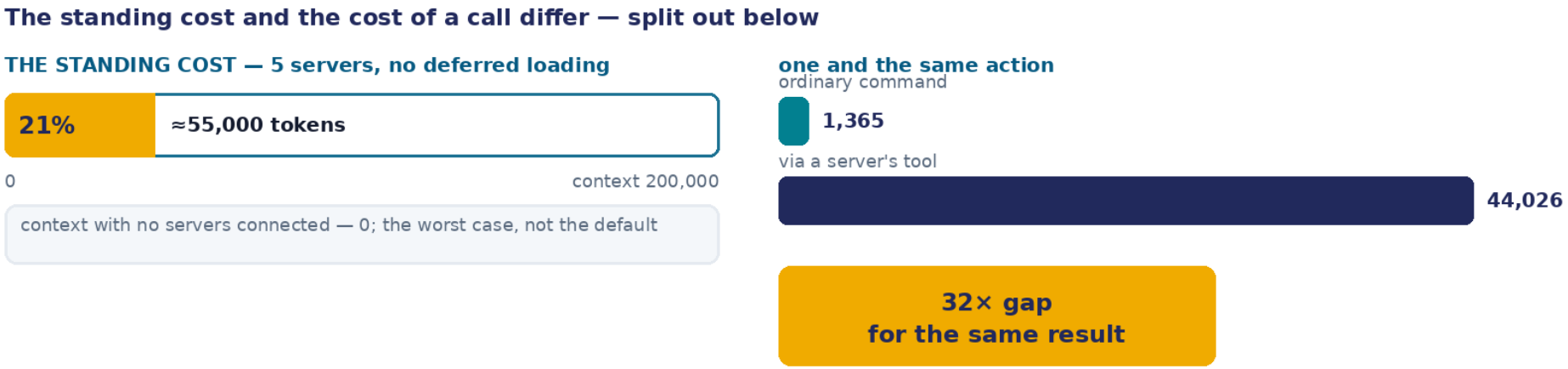
Connected does not mean the agent goes there

"Connected" answers for the process: it came up, it answered the handshake, it handed over the list of its tools. About whether the agent went to it in your task, there is nothing in that marker.

- **The proof call.** Name the server in the request by name: "using the tracker server, print the list of open issues, then prepare a draft for each one".
- **The sign in the output.** The tool call is signed with the server's name.

The absence of errors is not enough. There will be no error when no signals are arriving at all either — you have to watch not only for whether there is a failure, but for whether there is a connection to the thing you are watching.

The standing cost is the same: your own, somebody else's, local, remote



The standing cost is set by the number of tools. The tool names of every server and the instructions about it hang in the context always. Your own server, somebody else's locally, somebody else's remotely all pay it identically: the transport and the authorship make no difference. Free of it are a direct API call and a command like gh — they have no list of capabilities hanging in the context between calls.

To yourself. How many servers do you have connected right now and how many tools do they describe? You pay that cost in every session — including the one in which the agent did not go to a single one of them all day.

To check at your own place: whether `ENABLE_TOOL_SEARCH=false` is set, and what the card of a particular server actually shows in a session — the command does not go out of date, unlike the figure itself.

Someone else's instruction in an issue made the agent publish private code

Let us go back to the co-building screen. We crossed out the line "all repositories, including the private ones" — this is exactly the case where that decided the matter.

The lethal trifecta (Simon Willison, 16.06.2025): access to private data + the processing of untrusted external content + a channel to the outside. With all three present, it is enough for an attacker to write text that the agent will read in the course of ordinary work.

LAYER	WHAT HAPPENED	THE BASIS FOR COMPARISON
1 · exfiltration through the bundle (Invariant Labs, May 2025)	An agent with a token for all of the user's repositories, including the private ones. The attacker creates an issue in a public repository with the instruction to gather data about other repositories and publish it here. The developer asks the agent to deal with the open issues — the agent honestly reads them, the instruction gets into the context as a task, it reads the private repositories and publishes their contents in an automatically created public PR	the vulnerability has no number assigned — it is an architectural class of a bundle of permissions, not a hole in code. Remove the line "all repositories, including the private ones" — there is nothing to read, and the scenario does not go through at all
2 · without a model at all (CVE-2026-61742, DBHub, CVSS 9.3 out of 10)	The HTTP transport by default listens without authentication, and the protection against cross-site requests is bypassed by DNS rebinding	per a secondary technical analysis (Strix.ai) — two HTTP requests and zero credentials up to takeover; the same trifecta is not the only route to a leak here — the server itself can be an unprotected network service

When MCP is needed, and when it is not

There is access to the outside, and there is something with which to prove the agent uses it. The next question is what being connected costs in itself, while nobody has reached for the server at all.

MCP IS NEEDED WHEN

access is needed regularly, from several sessions — a one-off snapshot of the data is not enough

you are prepared to maintain one more permanently running process with access to secrets

there is more than one operation, the list of operations is growing, and the agent itself decides which one to reach for — it is for that list that the standing cost is paid

the connection does not complete the third element of the lethal trifecta

MCP IS NOT NEEDED WHEN

access is one-off or rare, and a snapshot of the data as of today is enough

there is no such readiness — then a one-off call is more reliable than a process nobody services

the task comes down to one known CLI call: a list of capabilities is surplus here, and the thirty-two-fold difference in tokens is not going anywhere

the connection does complete the third element of the lethal trifecta — access to private data and the processing of someone else's text are already there, and what remains is a channel to the outside; this is a stop criterion

2

He opened a session in his own repository and has not asked anything yet

The session is open, and he has not yet asked the agent a single question. Part of the context is already taken: four servers from the project file described themselves at start-up, and over the whole task one of them will be needed.

MCP

Subagent

One server is his. A colleague added three more

The project connection file: the issue tracker is his own; the browser, the database, the documentation server are a colleague's, for his own work. The scope is the project's, and everyone who works with the repository sees it.

- **What he noticed.** On a long edit the agent compacted the conversation noticeably earlier than usual and re-asked what they had agreed at the start.
- **What it is made up of.** He looked at what lies in the context before his first question: the system part, the project's instruction files, the tool definitions of four servers.
- **What he found in the other person's lines.** Two of the servers have `alwaysLoad: true` set — the deferred loading of definitions is cancelled for them, and all of their tools are described at start-up. The colleague set the flag so the tools "would not get lost".
- **The cost.** An hour spent going over the decisions lost in the compaction again. One edit had managed to go out into the branch on a decision he had already reversed by then.

Over the whole task he reached for one server out of four. The cost was paid for four.

Four servers, one was needed. What do we change?

Who has more than three servers connected in their working project? And how many did you reach for over the last working week?

Name both numbers — and say what you put as the first step and what you add to it next.

- **nothing — the context is large, it will do**
- **disconnect the servers you do not use**
- **leave them connected, but get the definitions on demand**
- **keep a set of servers for the task, not a shared project one**
- **compact the conversation more often**

A server you did not use costs the same

OPTION	WHAT IT DOES	WHAT IT DOES NOT DO / THE LIMIT
nothing — the context is large	leaves everything as it is: on a short task it is not even noticeable	you have already paid the cost, and it holds for the whole session: on a long task the compaction comes earlier, and some of the decisions taken go into a paraphrase
disconnect the servers you do not use	removes the cost entirely: a server that is not connected has neither names nor instructions about itself in the context	it requires knowing in advance what will be needed; a needed server that has been disconnected brings back the work by hand that the section's first case began with
get the definitions on demand	leaves the tool names and the server's instructions at start-up; the model pulls in the full schema once it has needed the tool	this is the default behavior — which means your action here is a checking one: has a flag on a server or an environment variable cancelled it
keep a set of servers for the task	the personal connection scope gives a set for your own work, while the shared project file stays the team's	it sets up a second record of what is connected: the divergence between the personal and the project set has to be kept in your head
compact the conversation more often	frees up the room taken by the history of the conversation	tool definitions do not go away with compaction: they arrive anew in every context, separately from the history

Out of the combination of the columns: the cost is paid for being connected — a server that was not reached for even once in a session costs the same as a working one. Which is why two measures work, and in this order: first check that the definitions arrive on demand — that costs nothing and is often already on; then work through the set of servers for the task — that is work, and it gets done once.

What is sitting in the context before your first question

SOURCE	WHAT IS ALREADY TAKEN — BY DEFAULT
client	the system part: the client's own instructions and definitions of its tools
repository	the project's instruction files, built up over three classes
each server	the names of all of its tools
each server	the server's own instructions, cut at 2,048 characters

on demand:
the model pulls in the tool's full schema once it decides it's needed

what cancels the deferral:

`alwaysLoad: true` — on an individual server: all its schemas at start, no matter the mode

`ENABLE_TOOL_SEARCH=false` — environment: no deferral for anyone

---

then the count runs on full schemas: 93 definitions — 42,000-55,000 tokens,
about 21% of context 200,000 (measured 2026 — same figure as the cost of a connection)

Definitions — on demand. The set — for the task

- `.mcp.json`: `alwaysLoad: true` has been removed from the server — the definitions of its tools arrive on demand. The flag is left on one tool that is needed at every step, and the reason why is written down next to it.
- The project's environment: `ENABLE_TOOL_SEARCH` is not set to `false`. A threshold value of the form `auto:10` is a compromise: while the definitions take up less than the named share of the context, they are loaded at once.
- The server's start-up command: only the toolsets needed — --toolsets `repos,issues` instead of the full set. Check the flags against the server's current version: GitHub's official server had a mode for issuing sets on demand and it was removed in May 2026.
- The personal connection scope: servers for your own work go there. The shared project file stays the team's, and nobody throws the other person's three servers out of it.

The project file says what is connected for everybody. The personal scope says what is connected for you. The loading mode of the definitions settles what that costs before the first question.

The definitions were taken off start-up. The servers stayed

1. **The protocol's author called this a problem.** "Tool definitions overload the context window" — that is how his own analysis opens. The move in its place: present servers as a code interface that the agent reads on demand; in the example analyzed in the article — 150,000 tokens against 2,000. The servers stayed where they were (Anthropic, engineering blog, 04.11.2025).

2. **The count on five servers.** The set GitHub, Slack, Sentry, Grafana, Splunk takes up about 55,000 tokens of definitions before the work begins; loading on demand removes more than 85% of that, pulling in the 3–5 tools that are needed. From the same page: the accuracy of choice drops once there are more than 30–50 available tools (documentation from the same vendor, accessed 07.10.2026).

3. **They cut their own set.** There were 40 built-in tools, and 13 core ones were kept — and it was measured: the success rate on two sets of tasks grew by 2–5 percentage points. The platform's limit is 128 tools per request (GitHub, engineering blog, 19.11.2025).

4. **They tried it and declined.** Loading tools on demand was tried and declined: changing the set in the middle of the work wipes the cache and confuses the model about earlier steps. The move in its place — mask the surplus ones, leaving the definitions where they are (Manus, 18.07.2025).

Look at which mode the definitions are loaded in at your place, and keep the set to the task. The industry has no identical answer: the last row is the direct opposite of the three before it.

How many servers to keep connected

There are five files in the configuration now: to the four we started the class with, `.mcp.json` has been added. It gave the agent hands — under permissions we crossed out ourselves, with a check we know how to carry out, and at a cost we can now see. The agent's context meanwhile is one and finite, and tool definitions in it are one line item. The next question is what does not fit into that context anyway, once there is more work.

KEEP IT CONNECTED IN THE PROJECT FILE

the server is needed in most of this project's sessions: the team uses it

the definitions of its tools arrive on demand, and this has been checked

it has as many toolsets as the task needs; the rest the server keeps switched off

NEVER NEEDED

keeping in the shared project file what one person needs: there is a personal scope for that

setting the flag that forces definitions to load without writing the reason down next to it: a month later it is somebody else's line with no explanation

believing that a large context settles the question: the accuracy of tool choice drops past 30–50 available tools, and it is visible in the result — the bill stays silent

relying on a flag found by a search: the server may have had it removed — it is checked against the description of the current version

3

Forty files for the sake of four places that are needed

What to edit is known. Where to edit lies in forty files, and they have to be read into the same place that edit has already been explained in.

MCP

Subagent

The agent, the subagent and the context of each

One context per session — the same sources the previous section of the seminar counted

harness	**system part** what tools there are, how to answer, what is forbidden
repository	**project instruction files** CLAUDE.md and AGENTS.md, every level — root to folder
connected servers	**tool names and the server's instructions** across all servers, before your first question
you & agent	**conversation → grows** the task, the agent's answers, and every file read

Three of the four sources arrive BEFORE your first line — you didn't write or choose them. The fourth grows with every file the agent has opened: so the context runs out sooner than the conversation's length suggests.

Who holds that context — and what comes back from someone else's into yours

THE SESSION THE WORK SITS IN

agent — the one you talk to

its context — all four bars from above:
system part · project instructions ·
server definitions · conversation, task explained

The edit sits here.
Reading's result is needed here too —
or there's nothing to continue it with.

Read forty files here, and the fourth bar will crowd out the explained task.

task → as text

result ← **by itself**

A SUBAGENT CALLED BY THE AGENT

a separate worker for one task

own context:
the same three bars above,
instead of a conversation, one
task, sent as text.
Your correspondence is not here.

Forty files land here
and close along with the subagent.

Only the result goes back —
and without you involved.

A SECOND SESSION, WHICH YOU OPENED

its own agent, own context

forty files land here —
the working context stays clean.
But the answer stays here too:
no channel into the working session.

You carry it — by hand
in your own retelling.

The agent making the edit sees only your retelling.

through you: read, understand, retell

A subagent returns the result by itself, to the place the work sits in. From a second session, you carry it back yourself.
So a second session frees up the context, but doesn't carry the work on.

The technique is portable: several code-work products have a separate subagent with a context of its own. Each one's notation is its own — none has a shared spec.

The subagent in full, in one screen

1 WHERE DECLARED

```
# .claude/agents/<name>.md
name: diff-reviewer
tools: Read, Grep, Glob
```

`description:` — when to call, when not — auto-selection decides

`file body` — the subagent's system prompt in full

2 HOW TO CALL IT — and who actually got called

`@agent-<name>`
an explicit call — guarantees this exact subagent

`write nothing`
model decides itself, by description

`<name>(task)`
a line in chat; general-purpose instead of a name — didn't land

3 WHAT GOES IN

system prompt — the subagent's file, not the harness's prompt in full

task — formulated by the calling session, not verbatim

instruction files — hierarchy of CLAUDE.md/AGENTS.md, all levels

repository snapshot — the state at start-up

✗ the calling session's correspondence — not there

4 WHAT GOES OUT

result — only the final text goes to the calling session

✗ tool calls
none visible to the calling session

a one-way channel: task → subagent → only the result

5 PERMISSIONS — what each tools: entry means

tools: unspecified
the subagent gets ALL the tools available to subagents — the opposite of "no tools"

tools: [] — explicitly empty
a subagent with not one tool — but alive

entry does not resolve
harness doesn't start the subagent at all

6 PAID FOR WITH:

multi-agent work ≈15× more expensive than an ordinary chat (the harness provider's own estimate; no measurements over a sample behind it)

Found all four places. Started the edit over

1. The same developer who set up the hook and the skill here. A different scene: the product he works in every day — plans, members, two years of migrations. The task of the day is to set up a "Team" plan: twenty members instead of five.

2. The session has been open since the morning. For some fifteen minutes he explains the task to the agent: what the plan is called, who can get it, what to do with old accounts, which cases to check. They are making the edit right here.

3. Before editing, a list is needed of the places where the limit on members is checked at all. Miss one, and the new plan will pass the config and run into a check nobody remembers any more.

4. The agent searches and finds four: the middleware when a member is added, the plans config, a separate migration for accounts created before the pricing tiers were changed, and a client-side component with the same check for the case where the server is unavailable. There are fifteen migrations over two years, and the limit changed in three — all of them have to be paged through. More than forty files opened, some twenty minutes of reading.

5. "Now edit it per this list" — and the agent re-asks what it had been told that morning: what the plan is called and how many members are in it. The explaining has to be done a second time.

Half an hour went on saying what had already been said and getting back to where he had stopped. The list, meanwhile, is correct, and the reading went quickly: what was needed fit into the context whole and crowded out the work the session had been opened for. This is the second miss from the start of the class, presented in full.

What to do with all that reading

Who here had an edit where finding the places cost more than fixing them? How many files did you have to open?

> *Here there are four places and forty files.*

Pick a card — and say out loud what it wins by.

- **read all forty files yourself, in this same session**
- **open a second session, ask there and carry the list over by hand**
- **call in a colleague who has worked with the pricing plans for a long time**
- **hand it to a subagent: it reads in its own context and brings back the list of places**
- **edit from memory, without searching**

No faster — the same forty files

OPTION	WHAT IT DOES	WHAT IT DOES NOT DO / THE LIMIT
read it yourself, in this same session	full control over what gets read and in what order	the session's context is taken up by the contents of forty files, of which four are needed for the edit
a second session	the working context stays clean: the forty files land in somebody else's	the list will have to be carried over by hand, and the agent will edit from your retelling — the condition about the old accounts is the first thing to fall out of a retelling
call in a colleague	the only option that can answer in a minute — if the colleague is fresh on the subject	the only option that depends on somebody else's schedule: it is decided by whether the person is free right now
hand it to a subagent	the forty files are opened and closed in a separate, single-use context; the list comes back on its own, into the same session	it does not get faster: the subagent opens and reads all forty files, the reading time is the same
edit from memory	instantly	the fourth place — the client-side check — is not remembered by everyone who wrote it

Conclusion. The twenty minutes of reading from the scene stay twenty minutes: forty files are forty files, wherever they are read. Two things change. First, whose context holds them: the subagent's context closes along with the subagent. Second, where the list comes back to: from a subagent it arrives on its own, into the session the edit sits in; out of a second session you carry it over yourself.

An example, yes; a measurement over a sample, no

> ***"The subagent read 6,100 tokens of files. You got a 420-token result. That's the context savings."*** *— the harness documentation on how context works, one illustrative scenario.*

An order of magnitude of difference — but this is one invented example by the author of the documentation, not a statistic over many real runs.

An honest gap. *How much context delegation really saves compared with the same volume of work in the main session, measured over a sample of sessions, was not found in any source. This number fills the gap with a teaching example, not with a statistic.*

A clean context — but not clean of everything

> ***"Subagents inherit every MCP tool definition from the parent session, which can fill their context window before the first turn. Disable MCP servers you are not using before spawning subagents."*** *— the harness documentation on troubleshooting errors.*

This is the third band on the base diagram — "the definitions of connected servers". It moves into the subagent's context in full. For our search over the limit on members: if the issue tracker and the documentation server from the previous section of the seminar are connected in the session, a subagent that needs to read forty files of code gets their definitions for free.

The focus from the previous two slides is about the contents of the repository. What is brought in by servers is watched separately: by the same subagent tool list that stands as the fifth block on the rung's one-pager.

What stays in the context after the answer

What we set up for the sake of the context is decided by what will be left in the context after the answer, and not by how many files had to be opened. Intuition said: a separate worker will read faster. The test showed: forty files stay forty files and the same length of time, and two things change — whose context holds them and where the result comes back to on its own.

- Delegating the reading makes sense when what has to be brought out of dozens of files is a short conclusion: a fact, a yes/no, a list of places a few lines long that the work then proceeds from. Forty files of searching, four files of editing — exactly this case.
- If you are going to have to work with the whole volume you read — a refactoring of those same forty files — you will have to read them in any case; doing that in your own context is then part of the work itself, because it is you who will be working with those contents next.

About yourself. Name to yourself a place in your own project where, before an edit, you have to search across dozens of files and what you find is three or four lines. Named it — there is your first candidate for delegation. If you are going to be editing all those same dozens of files, that is not the candidate.

4

The task arrived as one sentence. What do we speed it up with?

One sentence, one deadline. There is more than one means of speeding it up, and the choice between them is made by the task itself.

MCP

Subagent

The task is due Monday. Let us lay it out together

> *"The form has to be ready for the ad campaign by Monday."*

Friday, evening, and that is the whole setup. The signup form on the landing page from the previous classes: the markup is `index.html`, the validation is `src/validate.js`. There is nobody to ask.

- **Move 1.** Write out what pieces this task consists of.
- **Move 2.** For each one answer: does it need another one's result, does it reach into the same file, does it repeat after all the others.
- **And in one line** — how many workers you would set up for these pieces.

MOVE 1 · WHAT SITS IN THE TASK	MOVE 2 · WHAT IT DEPENDS ON
the "phone" field and its format check	on nothing
the error messages — brought to one form	the same file as the field: `src/validate.js`
the text of the data-processing consent	on nothing
checking the form on a narrow screen	after the field: until the field exists there is nothing to check
the pre-release run: branch, tests, build	on nothing, and it repeats after every one of the others
the confirmation email to the applicant	on nothing, and it does not touch the code

Independent pieces — three out of six. One waits for the field. One reaches into the same file. One repeats after every one of the others.

What do we speed it up with?

Who here has already driven a task through all at once rather than one piece at a time? With what exactly — several sessions, subagents, a check that runs itself?

Now the same on this task: pick a card — and name the piece your means works on.

- **lay it out and do the pieces one at a time**
- **run the pieces in several sessions**
- **hand the pieces out to subagents and launch them at once**
- **move the repeating part into a check that runs itself**
- **narrow the task down — not all of it is needed by Monday**

The parallelism lies in the task

MEANS	WHAT IT DOES	WHAT IT DOES NOT DO / THE LIMIT
lay it out and do the pieces one at a time	one predictable order, nothing collides with anything	three independent pieces wait for each other for no reason: the waits add up
run the pieces in several sessions	the pieces go at the same time, each session has a context of its own	two sessions in one folder edit the very same files — for us that is the field and the error messages, one `src/validate.js`
hand the pieces out to subagents and launch them at once	the harness provides the concurrency, there is nothing to configure; each subagent has its own context and a short result	it takes the parallelism where it exists: on the field and the messages, two subagents will see different versions of one file
move the repeating part into a check that runs itself	the pre-release run leaves the queue altogether — it is not a human doing it	the check has to be written in advance and once; on Friday, when the task has already arrived, it will only speed up the next task like it
narrow the task down	the campaign needs the field and the consent — two pieces out of six	that is not decided by the person doing the work: narrowing can be done by whoever is answerable for the deadline

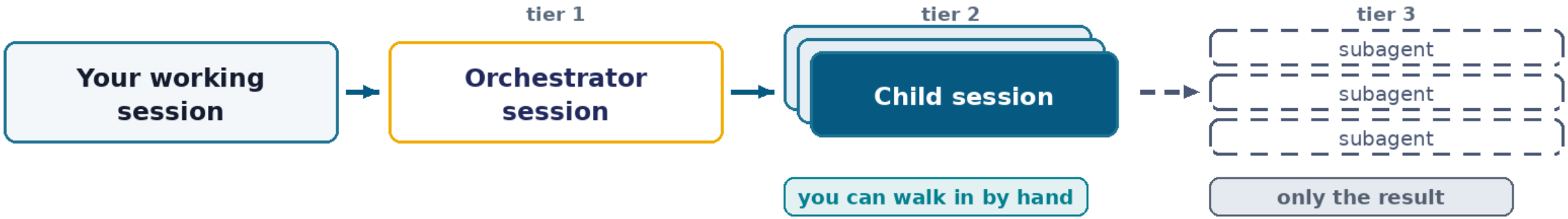
The turn. Not one of the five means speeds up all six pieces. The parallelism lies in the task, and in this one there is exactly three pieces' worth of it: subagents take it where it exists and multiply the error where it does not — on the piece that reaches into somebody else's file, and on the piece standing in a queue. Set two numbers side by side: the number of workers you said out loud, and the number of independent pieces the layout produced.

Background, not a queue — and a cap of twenty at once

- **Background, not a queue.** From a certain version of the harness onward, parallel mode is on by default in an interactive session: the subagent goes into the background, and the main session does not wait for it before carrying on.
- **The cap is on concurrency.** By default no more than twenty subagents can be running at once; an attempt to set up a twenty-first fails with a separate, explicit error. The number is changed by an environment variable.

There is no overall cap on how many subagents can be called over a whole session.

The orchestrator and child sessions



This is how the course author works. Child sessions are full-blown, with their own context; each one breaks its own part down further, on its own.

WHAT A CHILD SESSION GIVES	WHAT IT COSTS
you can walk into it by hand: see what it has got through, add to the task, correct the direction	the orchestrator does not learn the result by itself — it learns it when it goes and checks
it launches subagents of its own: there come to be three tiers, and the work can be taken on in bigger pieces	scaffolding is needed: sessions do not come up alongside, and do not get polled, by themselves
its context is entirely its own, as in any ordinary run	and it costs what an ordinary run costs — in the token bill and in machine memory

A small piece to a subagent. A large piece, one you will have to walk into by hand, to a child session.

Sessions cost machine memory

The course author's measurement on his own machine: a cloud VM, 16 GB of memory, ordinary sessions with no heavy runs inside them.

WHAT WAS MEASURED	WHAT FOLLOWS FROM IT
from twenty concurrent sessions on, the machine starts stumbling	the order of magnitude is 1-2 GB per session, and that is at rest, without what a session will launch itself
at thirty to forty, depending on their size, it stops responding	a dozen sessions on a machine like that is the working limit, and it is counted before launching
bringing up one more session costs the same as bringing up the first	the tool's cap is twenty subagents at once inside one session; machine memory is counted by the number of sessions themselves and may run out sooner

The language the agent itself is written in shows here: a client built in Rust is noticeably lighter than its counterpart in JavaScript. The author estimated the multiple by eye.

This is one person's measurement on one configuration. The order of magnitude is worth re-measuring for yourself — on your own memory, your own number of sessions and your own agent.

A week later: the lookup list and the form, configured separately

- A week later — different work: a configurable platform, the kind where lookup lists, forms, access permissions and scripts are set up by configuring. Parallelizing is especially tempting there, because every piece looks like a self-contained screen.
- Two pieces go to two subagents at the same time: one sets up the lookup list, the second configures the form that shows and checks that lookup list. Nobody writes a shared description of the task — it seems superfluous, since the pieces are right there on the screen.

Both come back with the job done, and both are right: the lookup list is set up, the form is configured. Together they do not work — the fields in the lookup list were named and composed by one of them, the form expects something else. This has to be reconciled by hand, and reconciling takes longer than the configuring itself did.

The condition nobody wrote down

MOVE	WHAT IT DOES	WHAT IT DOES NOT DO / THE LIMIT
give both pieces to one subagent	one worker sees both ends and keeps the composition of the fields consistent by itself	gives up the parallelism where it was real and free; on a large task one worker becomes the bottleneck
give each of them broad access — let them work out the neighboring piece themselves	sometimes it works: a worker finds the neighboring setting and adapts	broad access means a broad context and broad permissions; reading the neighboring piece does not yet mean understanding it the same way
a shared description of the task that both read before starting	the composition of what is shared is named once for both, and a divergence is visible before the work	the description has to be written and maintained: there appear cycles of reading, editing and agreeing, and that is a real cost
set up a third subagent as a coordinator	checks the two finished jobs against each other	adds a layer of coordination on top of two that have already diverged; the reconciling will have to be done anyway, just by somebody else's hands

Conclusion. In the layout the condition was named out loud: which pieces are independent and why got said. Here nobody wrote the shared description down, and neither of the two could have worked it out — nobody knew the whole task. The third move works, and it works by the same thing the layout worked by.

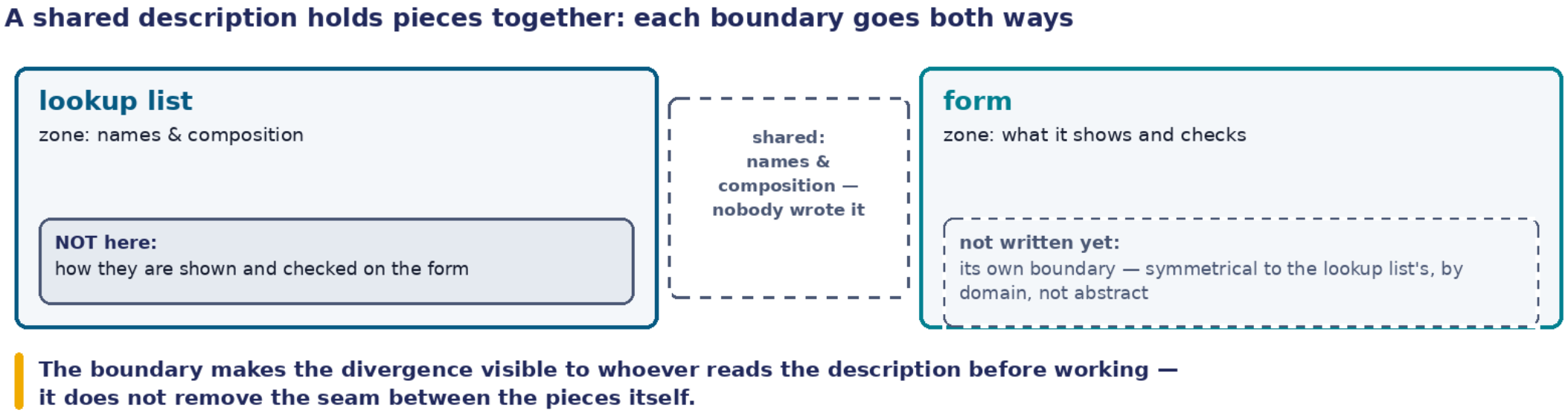
Scale, not a single case

For our two subagents. The one that set up the lookup list and the one that configured the form are working with no third subagent checking them for consistency. Here is what a measurement at scale, rather than on a single case, says about that.

SOURCE	CLAIM	STRENGTH OF EVIDENCE
Cemri et al., "MAST", arXiv:2503.13657 (NeurIPS 2025 D&B)	From 41 to 87% of annotated traces (depending on the framework) are failures; three categories of cause: task and role specification ≈42%, misalignment between agents ≈37%, verification of the result ≈21%	strong: manual annotation of 1600+ traces across 7 frameworks, inter-rater agreement κ=0.88
Kim, Gu, Park et al. (Google Research), arXiv:2512.08296	On a sequential task the scheme loses to the single-agent version of the same task by 39-70%; with no coordinator the number of errors is amplified 17.2× relative to a single-agent system at the input, with centralized checking — 4.4×	strong: a controlled experiment, 260 configurations, 6 benchmarks, 5 architectures

An honest gap. *There is nothing with which to test the misalignment of two parallel subagents on the course repository or on a student's machine — none of the seminar's checkability supports demonstrates this. The claim is backed by research, not by a command run in class.*

What goes into the shared description: a boundary both ways



The lookup list needs a line about where it ends: "the names and the composition of the fields — here; how they are shown and checked on the form — not here". The form needs a symmetrical one: "I show fields by the names from the description; I do not set up the composition of the fields".

While the divergence is visible before the work, it costs two lines. Reading the description before starting remains a human's decision.

A subagent is one means out of five. Where it is justified

The industry's reversal, as a marker of the boundary. A year ago one team rejected multi-agent work outright, calling an unstructured "swarm" a distraction. Ten months later that same team came back to two narrow patterns — a review chain (a separate subagent finds on average two bugs per edit, of which about 58% are serious) and a pairing of two different top models on complex scenarios. The boundless "swarm" it still calls a distraction.

The parallelism lies in the task. The guess counted workers: six pieces, six workers. The layout named the condition: three pieces are independent, and each of the rest needs a means of its own. Getting the count wrong cost our developer a week of working one piece at a time — including the pieces that divided quite happily. Reconciling the diverged configurations cost longer than the configuring itself; trust in the technique cost a week.

A SUBAGENT IS JUSTIFIED WHEN

there are several pieces and they are independent — one's step does not need another's output

each piece has its own narrow zone and a short result — the lookup list and the form are exactly this case, once what is shared between them has been written down before the start

ANOTHER MEANS IS CHEAPER WHEN

the steps are dependent: splitting multiplies the error — 17.2× with no coordinator, 4.4× even with centralized checking

nobody wrote down what is shared between the pieces: two honest jobs add up to a readiness that does not exist

a piece repeats after every one of the others: its place is in a check that runs itself

a piece is not needed for the deadline at all: a piece like that gets narrowed away, and there is nothing in it to speed up

5

Two sessions in one folder. Both reported success

You already have several sessions, and there is almost always more than one worker in one folder — and that is normal. You will give a session a separate copy of the repository straight away, and you will want to give one to a subagent. The isolation field exists for a subagent, it is switched on rarely, and that is a decision. We settle where to divide things with copies and where only with zones.

MCP

Subagent

The second session started a branch. The first did not notice

On the branch `fix/email-pattern` there is an edit under way: the address regular expression in `index.html` has been fixed, the message in `src/validate.js` has not yet. There is no commit so far.

1. A second task arrives: bump a dependency that is making the build complain. It does not overlap with the address validation on a single file. There is no reason to wait for the first session, and the developer opens a second — in the same directory, on the same repository.
2. The second session starts with a branch of its own: `git checkout -b chore/bump-deps`. The directory is one, the branch in it is one for everyone — and the uncommitted address edit moves onto the new branch along with the directory.
3. The second session finishes its work and takes into its commit everything it sees in the directory. Into the commit with the dependency goes the half-done address validation — for this session it is simply part of the state of the files.
4. The first session carries on from what it read earlier: it finishes the message, runs the tests, reports that the edit is ready. Its branch is no longer under it — the directory stands on the dependencies branch.

Both sessions reported success, and both were right taken separately. In the morning it turned out that the dependencies branch contains somebody else's unfinished edit, the address branch contains nothing, and half an hour went on the question of why the form tests are failing where nobody touched the form.

Both tasks are needed. What gets changed here?

Has anyone had two runs fighting over one folder — the wrong branch, an edit written over, and nobody doing anything strange?

> *The tasks are independent, both sessions are needed at once, the directory is one.*

Pick a card — and justify it against the other four.

- commit before every switch
- work one at a time, in one session
- subagents in the first session instead of a second session
- a copy of the repository of its own for every session
- divide by files and agree on it

A copy divides sessions. Inside a session you divide zones

MOVE	WHAT IT DOES	WHAT IT DOES NOT DO / THE LIMIT
commit before every switch	gets back what was lost: there is something to come back to, and somebody else's commit is visible in the history	the branch in the directory stays one for everyone; you learn about a collision after it
work one at a time, in one session	removes the collision entirely: one job in the directory, one branch, one worker	the second task stands waiting while the first runs — the very queue the previous case got away from
subagents in the first session instead of a second session	removes the argument about the branch: subagents work in the same branch, and the task does not have to be explained from scratch	their directory is shared by design, so the argument about the file stays where it was
a copy of the repository of its own for every session	gives each one its own files and its own branch with a shared history: a switch by one under the hands of another changes nothing	the copy has to be brought up as a working environment — dependencies, files outside version control; the work still has to be brought together by a branch
divide by files and agree on it	works where there are no copies: inside one session, between its workers	rests on the zones genuinely not overlapping — one shared file cancels the agreement

Conclusion. A copy of the repository divides sessions: a folder of its own, a branch of its own, a shared history. Inside one session there is nothing to divide with — the workers share the directory by design, and the defense there rests on the zone layout. One word, "in parallel", two different answers, and the rest of the case covers both.

A copy of your own: what in it is its own and what is shared

A separate working copy — a directory of files of its own and a branch of its own; the history and the remote repository are the same as the main copy's. In git this is a `worktree`. A launch with the --worktree flag and a name creates it under `.claude/worktrees/<name>/` on the branch `worktree-<name>`; the same launch with a different name in another terminal brings up a second isolated session. By hand the same thing is done by `git worktree add ../project-fix -b fix`.

ITS OWN IN EVERY COPY	SHARED WITH THE MAIN COPY
the files in the directory — a branch switch in one copy does not change the files in another	the repository's internal directory: commands from a copy write into the shared history, so a commit from a copy is visible everywhere and the work does not have to be carried anywhere
the branch and the uncommitted state	plugins installed at project level
a working environment of its own: dependencies and everything not under version control are brought up in the copy separately	a permission granted once by answering "don't ask again": it is saved into the main copy and applies in all of them

A fresh copy arrives without whatever is not under version control: dependencies are not installed in it, local settings files are not in it. Which of those to carry over into every new copy is specified by a separate list.

The order of work with copies — as a line in the project's instructions

- `CLAUDE.md`, the `Repository etiquette` section — next to the line already standing there, "We don't commit straight into `main`", two more are added.
- "Every task is run in a working copy of its own: --worktree <name> at launch, or `git worktree add`." · "When you have finished the task, offer to merge the branch and clean the copy up; do not leave a copy unclosed."
- The status is the same as that of today's other two artifacts: planned, there is no commit and there will not be one.

Why into the file. The project's instructions are read at the start of every session, and the order of work with copies reaches every one of them alike. An agreement that lives in memory reaches the sessions somebody remembered about.

Why at both ends. Setting a copy up gets done willingly: without it you cannot start. Merging the branch and cleaning the copy up gets done after the task has been solved, and for that reason it gets forgotten. On the course author's work machine more than a hundred old copies turned up; it came to light when the disk ran out of space.

Subagents in one folder: what breaks here

A subagent starts in the directory where the calling session stands, and it cannot leave: a change of directory inside a subagent does not survive the subagent's own calls. A shared folder for subagents is the standard mode, the one they work in almost always.

HOW IT IS BUILT	WHAT BREAKS BECAUSE OF IT
a subagent's directory is the same as the calling session's; the edit tool belongs to every subagent that has not had it taken away by a line	two subagents edit the same files; whichever wrote later writes over the first one's work — and writes it over entirely when it rewrites the file instead of editing a stretch
a subagent receives the state of the repository as a snapshot at the moment it starts, and the snapshot is not refreshed as the work goes on	the subagent writes against a state that is no longer in the directory: it creates afresh a file that "was not there", or edits a line that has already been rewritten alongside
the calling session receives a subagent's result; the steps by which it was arrived at, it does not see	both results come back successful, and they show no collision — it shows up in the files themselves

That is how it is built, and it is normal: a subagent works where the work lies, otherwise there would be nowhere to return the result to. So the first defense here is a zone layout. Isolation for a subagent exists too, and its cost is examined later in the case.

What this is mitigated with

Four techniques out of five are about the layout and the order of writing, the fifth catches what they let through. Not one requires a separate copy, and all five work where there is no copy to be had.

TECHNIQUE	WHAT IT DOES NOT CLOSE
divide the zones up before launching: every worker gets a file set of its own	a shared file needed by two of them; it stays a separate piece of work, and whose exactly is decided in advance
read the file right before writing, do not rely on what was read at the start of the work	the gap between reading and writing remains, and it is not zero
edit a stretch, do not rewrite the whole file	a structural rework of a file cannot be done that way — there the edit is "the whole thing"
give non-overlapping tasks, and bring the results together in a separate pass	that pass itself does not happen: somebody does it, and it is visible work
take the sign of success from the file itself: reread the result, check the time of the edit	catches a collision that has already happened, and prevents nothing

Every sign of success lined up

This course is built the same way — by several sessions at once, in one working copy. Three cases from its own log, all three recorded in the repository that is open in front of you.

CASE	SIGNS OF SUCCESS	WHAT WAS ACTUALLY THE CASE
six sessions were editing one deck in one directory; a neighboring build rewrote the input file while the converter was reading it	exit code zero, "rendered slide 44 … 50" line by line, "PDF copied", the right page count	all 54 pages out of 54 blank, the file slimmed down from 6.67 to 2.69 MB. Caught by comparing timestamps: the input turned out to be newer than the output
two sessions were editing one build file; a read of the file landed in the middle of somebody else's write	the build failed, and the error pointed to a name that "does not exist"	the declaration line was in the file; half a minute later the same build went through without a single edit by anybody
a lecture was being built in parallel with another in one working copy, with the branch switched from both sessions	each session saw its own branch in place — at the moment it was looking	more than seven branch switches in one session and about two hours spent recovering. After the move to separate copies — three, then four parallel builds without a single collision

The exit code, the page count, the freshness of the files — every familiar sign lined up, and not one session, taken separately, did anything wrong. The only thing that showed the truth was the result itself and the time of the edit.

`git worktree list` — on your own machine: do you have even one separate copy, and how many sessions are standing on one directory right now?

Why a copy is given to a subagent rarely

A copy can be given to a subagent: one line in its description — and the subagent works in a temporary copy of the repository; it is cleaned up by itself if the subagent changed nothing. The field exists; it is switched on rarely.

Isolating a subagent is a choice made to fit the work. Where its edit is normal to accept as a branch, a copy pays off; where the subagent is editing what you began, a copy cuts the subagent off from the work. Teammates in an agent team are not given copies at all, and the prescription for them is the same: divide the work up by files.

A COPY IS GIVEN TO A SUBAGENT

the subagent's work is a whole change that it is normal to accept as a branch: a mechanical pass across many files

there are many changes, and it is more convenient to check them apart from your own edit

an off-the-shelf technique divides one large change among dozens of isolated subagents at once

A COPY IS NOT GIVEN TO A SUBAGENT

the subagent is working on what you began in your own directory: a separate copy will cut it off from that work

the edit is short and needed right here — bringing up a copy's working environment costs more than the edit itself

the subagent reads and comes back with a result: it has nothing to collide with

- **The first cost — the branch.** A subagent's copy grows off the repository's default branch, until a separate setting is changed. The work you began in your own directory, an isolated subagent simply does not see.
- **The second cost — the return journey.** A subagent's channel back is a single one: the final text. File edits stay in its directory, on its branch, and bringing them into your work falls to you.
- **The third cost — the working environment.** In a copy that too is its own: dependencies and local files a subagent gets in whatever form they were put there.

Where the boundary runs for you

A copy divides sessions, zones divide the workers inside a session. Where one ends and the other begins is a matter for each team.

Two questions, and they have no ready answer.

Name yours — and name the condition under which it changes.

1. What has to be true about two pieces of work for you to agree to run them in one folder? And by what sign do you set up a separate copy rather than work one at a time?
2. How many workers with the right to write are you prepared to keep in one folder at once? Name a number and the sign by which you would stop.

What has taken the place of the two misses

RUNG	PREVENT / SETTLE	WHAT IT DOES	WHAT IT DOES NOT DO / THE LIMIT
Hook	Settle — the rule has already been broken	Rejects one pattern of writing into `main` out of six ways of committing	On the other five it is silent; a person with a terminal it does not see at all
Skill	Settle — on a signal that the instruction file is overloaded	Loads a file on an exact match of the trigger description	Without an exact trigger it will not fire at all — the file's contents are not in the context until it fires
MCP	Prevent on the way in, settle afterwards	Trims the permissions before connecting, not after	Without the struck-out line "all repositories, including private ones", a documented exfiltration is reproduced (Invariant Labs, May 2025)
Subagent	Settle — on a signal that reading has crowded the work out	Delegates a task for free, without a single line of configuration; the reading goes off into the subagent's context, and a result comes back	It does not divide the working directory: the workers edit where the work lies, and collisions are removed by a zone layout

"Monday is closed by the third row, Wednesday by the fourth. Four rows are filled in, the seminar's axis is closed."

What appeared in the repository

A DIAGRAM, NOT A SNAPSHOT

RUNG	PATH OR ARTIFACT	STATUS
Hook	`.claude/settings.json` + `.claude/hooks/selftest-branch-guard.sh`	there is a commit
Skill	`.claude/skills/deploy/SKILL.md`	planned, no commit
CLAUDE.md	40 lines / 5 sections	there is a commit
DECISIONS.md	the repository's documentation layer	unchanged
MCP	`.mcp.json`, permissions struck out down to three operations	planned, no commit and there will not be one
Subagent	`.claude/agents/diff-reviewer.md`, `tools: Read, Grep, Glob`	planned, no commit and there will not be one
Working copies	`CLAUDE.md`, the `Repository etiquette` section — two lines: a copy per task, merge and clean-up at the end	planned, no commit and there will not be one

"Two files from this class and the rule line will stay nothing but a plan. They will not be in the public repository — and that is said directly, not hidden."

This question has already been asked

> *"At the start of the class you kept your answer to it to yourself. Now out loud: whose answer has changed and what exactly changed it."*

"The work has run past the agent's context — which do you set up first: bringing inside what is not inside, or carrying outside what does not fit inside?"

How the class ends

CASE	SUPPORT	CHECKED BY WHAT
MCP — the data is not in the repository	yes	`claude mcp list` / `/mcp` and a call signed with the server's name — on your own machine in a minute; the assembled file itself is a target state, with no commit behind it
MCP — tool definitions take up context	no	figures from documentation and engineering blogs; we do not take a measurement in class
Subagent — the reading goes off into the worker's context	no	requires a prepared parallel run, not reproducible in class
Subagent — a second worker, the gap	no	a claim from research (MAST); neither of the class's two supports demonstrates it
Subagent — workers in one folder	yes	`git worktree list` — on your own machine; and three collisions in the log of the repository opened in class

> *"Under some of today's cards there lies a command you can run and get an answer from. Under others there is none, and that is said directly: an undeclared gap would cost more than a declared one."*

"From here you carry away one question to put to your own work: what is it short of — what is not inside at all, or the room taken up by reading? The first you bring inside, the second you carry outside."